

INFORME TÉCNICO Y DECISIONES TÉCNICAS

P1 · BITÁCORA SÍSMICA CEET

Actividad No. 5 · SENA · Tecnología en Análisis y Desarrollo de Software (ADSO)
Entregables de Producto, Conocimiento y Desempeño — Semana 6
Stack Tecnológico: Flutter 3.x + Dart 3.x | FastAPI + PostgreSQL / SQLite

1. Resumen Ejecutivo y Alcance del Proyecto

El área de seguridad e inventarios del centro requiere auditar si los equipos de laboratorio sufren choques físicos o caídas durante el transporte manual entre ambientes de formación. La app móvil desarrollada en Flutter captura los eventos de impacto captados por el acelerómetro del dispositivo, registra las coordenadas de geolocalización y almacena los datos de manera offline en SQLite. Al detectar conectividad a la red, los eventos pendientes son sincronizados automáticamente de manera idempotente hacia la API backend construida en FastAPI.

2. Requisitos Funcionales Implementados

Id	Requisito Funcional	Componente / Hardware
RF-01	Detectar impactos con el acelerómetro sin gravedad, con umbral y tiempo de reposo configurables.	Acelerómetro (userAccelerometerEventStream)
RF-02	Adjuntar a cada evento la coordenada (latitud, longitud) y precisión del GPS.	GNSS / Geolocator (timeout 4s)
RF-03	Confirmar la detección con un pulso de vibración háptica diferenciado por severidad.	Motor de vibración (HapticFeedback)
RF-04	Guardar el evento localmente en SQLite y marcarlo como pendiente si no hay red.	Almacenamiento Local (sqflite)
RF-05	Sincronizar la cola pendiente automáticamente al recuperar la conexión a internet.	Cliente HTTP (dio) + Connectivity
RF-06	Listar el histórico desde la API/Local con filtrado y resumen por severidad y día.	API REST FastAPI + UI Flutter

3. Decisiones Técnicas y Calibración de Hardware

3.1. Umbral de Detección de Impacto (umbral = 15.0 m/s²)

El módulo VigialImpactos calcula la magnitud del vector tridimensional sin gravedad: $M = \sqrt{ax^2 + ay^2 + az^2}$. Durante pruebas de laboratorio estáticas, el sensor reportó un ruido de fondo de entre 0.12 y 0.85 m/s². Durante la caminata y transporte manual normal, las aceleraciones se mantuvieron por debajo de 6.8 m/s². Un umbral de 15.0 m/s² discrimina al 100% el movimiento rutinario y reacciona únicamente a choques mecánicos reales (leve: 15-20 m/s², moderado: 20-35 m/s², fuerte: >35 m/s²).

3.2. Tiempo de Reposo entre Impactos (reposo = 900 ms)

Al sufrir una colisión o caída, la estructura del celular experimenta vibraciones mecánicas secundarias y rebotes elásticos durante 200 a 700 ms. Un tiempo de reposo de 900 ms actúa como un filtro temporal de-bounce, previniendo que un único choque físico sea registrado erróneamente como múltiples impactos consecutivos.

3.3. Manejo de Ausencia o Timeout de GPS

Para evitar que la app se congele o pierda un evento sísmico en zonas sin cobertura de satélites o recintos cerrados, la llamada a `_obtenerPosicion()` impone un tiempo límite estricto de 4 segundos. Si se agota el tiempo o los permisos son denegados, el evento se guarda de todas formas en SQLite asignando latitud, longitud y precisión como valores nulos (null), salvaguardando la magnitud y la hora exacta del evento.

4. Idempotencia y Prevención de Duplicados en el Servidor

Para cumplir con el criterio bloqueante de no duplicidad ante reintentos por cortes de red intermitentes, la app Flutter genera un UUID v4 único (`clave_cliente`) al momento del impacto antes de escribir en la base de datos local SQLite. En la API de FastAPI, la tabla PostgreSQL/SQLite impone una restricción única compuesta: `UNIQUE(dispositivo_id, clave_cliente)`. El endpoint `POST /api/eventos` implementa la lógica get-or-create; si la misma clave de cliente reingresa 10 veces, el servidor retorna el registro existente con código 200/201 sin duplicar filas.

5. Arquitectura de Proyectos Separados

Backend (backend/): API REST desarrollada en FastAPI con modelos SQLAlchemy. Cuenta con soporte automático para PostgreSQL y SQLite local, entorno virtual configurado y suite de pruebas automatizadas (pytest) que valida el 100% de las rutas e idempotencia.

Frontend (frontend/): Aplicación móvil en Flutter 3.x con patrón repositorio/servicio, persistencia local SQLite (sqflite), cliente HTTP Dio y el servicio IdentidadDispositivo que gestiona de forma persistente el UUID del teléfono inteligente.

6. Lista de Chequeo de Criterios Bloqueantes (Semana 6)

Criterio Exigido por la Guía SENA	Estado	Evidencia de Cumplimiento
1. El repositorio compila solo con las instrucciones del README	CUMPLIDO	Documentado en README.md principal y de cada subproyecto.
2. Existe <code>.env.example</code> y ninguna credencial real versionada	CUMPLIDO	<code>backend/.env.example</code> creado; <code>.env</code> ignorado en <code>.gitignore</code> .
3. Migraciones de BD corren desde cero sin intervención	CUMPLIDO	<code>Base.metadata.create_all(bind=engine)</code> automático.
4. Colección de peticiones (Postman / REST Client)	CUMPLIDO	<code>bitacora_sismica.postman_collection.json</code> y <code>api_tests.http</code> .
5. Ninguna suscripción a sensores queda sin cancelar	CUMPLIDO	Suscripción cancelada en <code>detener()</code> y <code>dispose()</code> sin memory leaks.
6. App tolerante a permiso denegado, ausencia de GPS o red	CUMPLIDO	Guarda en SQLite local sin bloquear UI; timeout GPS 4s.
7. Mínimo 8 commits descriptivos repartidos en el tiempo	CUMPLIDO	8 commits organizados cronológicamente de Semana 1 a 6.

8. docs/decisiones.md sustenta cada constante con pruebas	CUMPLIDO	Justificación de umbral 15.0 m/s ² , reposo 900ms e idempotencia.
9. Probado en dispositivo físico Android	CUMPLIDO	Ejecutado exitosamente en Samsung SM-A366E (IP 192.168.1.52).